# Inresol Genius/BMS firmware notes

## Table of Contents


1. Installing and working with the ATxmega toolchain....1
   1.1 Compiling with ImageCraft....1
   1.2 Programming and debugging via AVR Studio....1
   1.3 Signing and direct firmware download....2
   1.4 Monitoring with the configuration tool....2
2. Code organization....2
   2.1. The service interface....3
   2.2. List of modules....3
3. Example tasks....5
   3.1. A/D monitoring....5
   3.2. PWM regulation....7


This document is intended as a short guide to getting-started with the *Inresol BMS* & *Genius* firmwares based upon the same *ATxmega* microcontroller, toolchain and codebase.

It is *not* intended as a full documentation for the code but as a starting point and refresher.

History

| Revision | Description | Author | Date |
|---|---|---|---|
| 0 | First draft | Johan Forslöf | 22/01/15 |

## *1. Installing and working with the ATxmega toolchain*

### 1.1 Compiling with ImageCraft

The *ATxmega* project are compiled with the *ImageCraft* C compiler. This compiler and software environment was selected to help jump-start the project and take advantage of code modules, tools and experiences from previous projects at *EDAB*.

A time-limited trial version of *ImageCraft* may be downloaded at www.imagecraft.com/devtools_AVR.html.

To compile a sub-project simply open the appropriate `.cbp` file and use the build function.

*Note: At the time of writing (2015-01-22) the present compiler version 8.15 crashes during linking, and the previous 8.13 release needs to be used instead (after taking care to first fully uninstall the later version).*

### 1.2 Programming and debugging via AVR Studio

The *AVR Studio 6* software suite contains the debugger and programming tools used by the projects. This software may be freely downloaded from www.atmel.com after registration. The version used at the time of writing is *6.2.1153*. The programming device used is the *Atmel AVR Dragon*, needs to be connected to a 6-pin PDI header on the boards.

To program and step through a newly built firmware use the *"Open object file for debugging.."* option in *AVR Studio* to open the `.cof` corresponding to the project. This may be located within the build output directory. When asked select the *ATxmega64A4U* MCU.

You should now be able to debug the project, with programming performed automatically at the start by default. Once this has been accomplished the *AVR Studio* project may be reused for future sessions.

The "*Device programming"* dialog may also be used for testing the communication against the device, and directly programming/reading-out memory areas, etc.

*Note: AVR Studio* appears to have difficulty with parsing `.cof` images of projects placed in folders with long pathnames or pathnames containing special characters. In case of difficulty opening an object file try moving and recompiling the project in the root of a drive.

## 1.3 Signing and direct firmware download

An alternate means of programming the MCUs while not to debugging is to prepare a pre-baked *HEX* image for programming. To this end a set of *"Sign <X> Firmware.bat"* scripts are used to generate, and fill in the required header fields, of these images.

Note that this links both the main application and the bootloader sub-projects together, which must first have been compiled via *ImageCraft*.

Once the *HEX* has been prepared the accompanying *"Download <X> Firmware.bat"* script may be used to download an image onto a device. In addition a prepared HEX image may also be downloaded through the bootloader via the *Config Tool* utility.

This programming script depends on the *atprogram* utility and drivers from the *AVR Studio* installation. Please register the *atbackend* folder of the installation in the `PATH` of the computer before proceeding.

*Note:* In addition to the firmware code a previously unprogrammed MCU also needs to have its configuration and protection fuses set. This task is performed by the download script but *not* by the debugger download.

## 1.4 Monitoring with the configuration tool

A PC service application allows the device to be monitored and configured, and reprogrammed via an on-board USB port. This system is accessed through the supplied *ConfigTool* application.

A user's manual for this application is beyond the scope of this document. However note that after reprogramming a device by other means than the bootloader the protocol descriptors may need to be re-read manually through the *"Upload all data"* button.

*Hint:* Press `CTRL+I` to access normally hidden features and fields such as the debug terminal.

# *2. Code organization*

The code for the *BMS* and *Genius* projects is written in standard C, with a limited attempt at abstracting the hardware. The code is further divided into a common framework section, as shared by both devices, as well as project-specific modules and definitions to fill in the specifics.

These relatively simple applications have straightforward timing requirements and a fully-fledged real-time operating system was deemed to be of limited benefit relative to the added complexity. Instead the main loop continually spins through and updates each sub-system on every cycle, with the occasional use of state-machines and interrupts for breaking up blocking tasks.

The convention is for the modules to provide a *Run* function which gets invoked during the update loop described above. Furthermore a few other standard interface functions are frequently exposed, such as *Init* for one-time initialization, *Idle* for checking readiness to sleep, a *OnParameterChange* callback for reacting to configuration changes, and so forth.

Another pattern is the division into generally applicable modules and the project-specific details, the latter denoted by a *Set* suffix. For instance the *ADC* module is responsible for performing general channel scanning whereas the set of channels to be measured would be listed in *ADCSet.*

A further convention is the naming convention of marking functions and variables considered internal to the module, yet which for technical reasons must be exported, with a leading underscore. These will frequently be provided with simple accessor macros in the header files as a concession to C compiler lacking inlining features.

One wrinkle is that the 8-bit microcontroller chosen has poor support for floating-point arithmetic. The application largely makes us of (mostly decimal) fixed-point numbers for basic arithmetic, resulting in some juggling of integer sizes and exponents to balance the range and processing requirements. In low-frequency processing tasks the 32-bit IEEE-754 floating-point library offered by the compiler should typically be preferred unless high precision is required.

## 2.1. The service interface

The central system which the code orients itself around is a service interface used by an external PC application to monitor the system. In essence any device exposes some set of tunable parameters, meter displays and error conditions. The specific set of of values exposed varies from application to application but the items within each category may be presented in a similar way.

Once the PC-based service application connects to a device it is responsible for interrogating the device about the set of exposed values, asking it to report on the the number of parameters, their names, values and other attributes. Internally each firmware stores a descriptor array defining this information and which the developer is expected to keep in-sync with actual parameter uses.

## 2.2. List of modules

Below is a brief list of the present software modules and their respective roles in the system.

| *Module* | *Description* |
|---|---|
| `ADC` | The basic A/D scanning loop, gathering and super-sampling the raw results. |
| `ADCSet` | A list of the channels to be scanned and their sampling settings. |
| `Analog` | Scale and post-process the resulting A/D measurements. |
| `Application` | The main application state-machine. |
| `BootMain` | Main function and application-specific adaptation used in the bootloader. |
| `Baudrate` | A table of predefined baud-rate settings for the RS485 and serial debugging ports. |
| `Bignum` | Multi-word integer arithmetic used for accurate charge tracking. |
| `Checksum` | CRC checksum calculation for protocol/EEPROM validation. |
| `Common` | Common types, utility macros, and hardware definitions. |
| `Comparator` | *BMS-specific:* Continually compare the current against limits programmed via the DAC channels, e.g. to quickly switch off on a short-circuit. |
| `Config` | Compile-time configuration variables. |
| `CPU` | Basic CPU state management (interrupt masking, bank switching, reset, etc). |
| `Ctrl` | Assign I/O pins and peripherals to the corresponding board function and modules. |
| `Eeprom` | Persist a memory block in the EEPROM with integrity checks and redundancy/wear-levelling |
| `EepromNVM` | Low-level primitives for writing to the internal EEPROM |
| `Error` | Track error conditions reported by the system, handling hysteresis, the effects of errors and so on. |

| | |
|---|---|
| `ErrorSet` | Define the specific array of error conditions implemented by a particular device. |
| `LED` | *BMS-specific:* Control the RGB LED. |
| `Main` | Initialize the system and run the main update loop. |
| `Meter` | A matrix of measurements and states presented to the user via the service protocol. |
| `MeterSet` | Define the specific matrix of meters implemented by a particular device. |
| `Monitor` | Gather up-to-date measurements and log data to be monitored by a user via the service protocol. |
| `NTC` | *BMS-specific:* Compute real temperatures from NTC measurements with pre-calculated tables generated by a separate utility. |
| `Numeric` | Format integers for display. |
| `NVM` | Access to the non-volatile information within the MCU, primarily the unique identifier. |
| `Oscillator` | Start/stop the main CPU and USB clocks. |
| `Pack` | *BMS-specific:* Overarching communication state-machine responsible for monitoring the cell states. |
| `PackDevice` | *BMS-specific:* Configure and read out cell status for a device on the BQ bus. |
| `PackLink` | *BMS-specific:* Datalink layer exchanging packets on the battery monitoring bus. |
| `Parameter` | Store an array of dynamically user-configurable parameters. |
| `ParameterSet` | Define the specific array of parameters implemented by a particular device. |
| `PWM` | *Genius-specific:* Program the high-frequency PWM module used by the buck-boost converter. |
| `Random` | Generate pseudo-random numbers, with entropy fed in from the ADC. |
| `Reset` | *BMS-specific:* Assert the hold-up signal keeping the supply voltage alive early on during initialization before the main function has been entered. |
| `RS232` | *BMS-specific:* RS232 serial port for debug trace output. |
| `RS485_Comm` | Data-link layer receiving and transmitting packets via the HLDC-based RS485 communication protocol. |
| `RS485_Proto` | Higher-level RS485 communication layer to be used in coordinating the BMS and Genius boards. |
| `Service` | Implement the custom self-descriptive communication protocol used for monitoring the system from a PC application. These packets are in turn wrapped by the USB stack. |
| `Service_Strings` | Expose a set of text strings, such as the serial number and build date, accessible via the service protocol. |
| `Shutdown` | Cleanly switch off the MCU while preserving the configuration and detecting unexpected resets. |
| `Sleep` | Coordinate putting the MCU into a sleep mode with a standard mechanism for avoiding race-conditions against the wake-up interrupts. |
| `SleepSet` | Application-specific set of wake-up/keep-alive conditions to be negotiated by the module. |
| `StateMachine` | Convenience functions for defining simple state-machines. |
| `Switch` | *BMS-specific:* Control the output switching by running the charge pump for the NMOS gate driver and allowing interrupt-safe stops on error conditions. |
| `Timer` | Application timing functions for defining interval timers and time-outs. |
| `Translation` | Map enumeration values into displayable strings for the service protocol. |
| `TranslationSet` | Define the application-specific set of state translations implemented by a specific device. |
| `USB` | USB stack implementing a HID device communicating with the PC service |

| | |
|---|---|
| | application. |
| `Watchdog` | Wrapper for the watchdog timer for insuring that a set of periodic events to continually occur, otherwise a reset is forced. |
| `WatchdogSet` | Application-specific set of periodic events to be monitored by the module. |

## *3. Example tasks*

Below are a pair of step-by-step examples of accomplishing illustrating how to perform common tasks within the system.

### 3.1. A/D monitoring

The theoretical task is to include, display, and process a new A/D sensor for measuring the 5V supply on the BMS board. In the example the sensor is connected to pin #3 (PA7/ADC7).

The ADC is configured to use the internal 1.0V reference voltage in the conversions. Therefore to suit this range the input voltage is first scaled through a 10k/47k divider. If required it would be possible to increase the reference voltage but beware that the *ATxmega* series does not support measuring inputs up to the full VCC rail.

The first step is to bind the new pin to the ADC7 channel for the benefit of the rest of the system. In *Ctrl.h* include the following line*:*

```
#define CTRL_ADC_5V ADC_CH_MUXPOS_PIN7_gc
```

Furthermore the pin, PA7, needs to be reconfigured as an analog input. Apply the following changes in *Ctrl.c*:

```
// PA7: 5V voltage (analog input)
PORTCFG.MPCMASK = BINARY(10101111);
PORTA.PIN0CTRL = PORT_OPC_TOTEM_gc | PORT_ISC_INPUT_DISABLE_gc;
PORTA.PIN4CTRL = PORT_OPC_TOTEM_gc | PORT_ISC_INPUT_DISABLE_gc;
PORTCFG.MPCMASK = BINARY(01000000);
PORTA.PIN6CTRL = UNUSED_PIN;
CTRL_PORTA.OUT = BINARY(00000000);
CTRL_PORTA.DIR = BINARY(00110000);
```

The new channel now needs to be included for measurement in the A/D scanning loop. Include the record channel in the enumeration listing the channel set in *ADCSet.h:*

```
ADC_RAIL,
ADC_5V,
```

In addition the new channel must also be included in the ADC_Channel array, in the same order, in *ADCSet.c*:

```
// ADC_RAIL
{
        /* MuxCtrl */ CTRL_ADC_RAIL,
        /* Ctrl    */ ADC_CH_INPUTMODE_SINGLEENDED_gc
},
// ADC_5V
{
        /* MuxCtrl */ CTRL_ADC_5V,
        /* Ctrl    */ ADC_CH_INPUTMODE_SINGLEENDED_gc
},
```

The channel will now continually be measured and should be visible as part of the ADC array in the

meter list. The raw value relative to *ADC_VREF*, a fixed-point integer in units of 100µV, may be accessed via *ADC_Get(ADC_5V)*.

The next step is to process the result and cancel the divider in the *Analog* module intended for processing incoming A/D results. First include an accessor for the voltage in *Analog.h:*

```
/* int16_t Analog_Get5V(void); */
#define Analog_Get5V() \
(CONST_CAST(int16_t, _Analog_5V))
extern int16_t _Analog_5V;
```

Next include the newly exposed variable in *Analog.c*:

```
// 5V supply
int16_t _Analog_5V;
```

Furthermore include a filter buffer in the structure  below. These are simple first-order low-pass filters which may be tweaked to reduce noise sensitivity.

```
Analog_Filter_t V5;
```

Now add the code for capturing and scaling the samples to *Analog_Run*. Here the previously mentioned divider is accounted for, and the filter constant parameter borrowed from the 3.3V supply measurement.

```
// 5V supply voltage
// · R? = 47k
// · R? = 10k
value = Analog_UpdateFilter
(
        &Analog_Filter.V5,
        ADC_Get(ADC_5V),
        P(3V3_FILTER)
);
value -= Analog_GetGND();
_Analog_5V = value * (47L + 10) / (10 * 10);
```

In single-ended mode the ADC has a slight ground offset which the code attempts to measure and cancel through the subtraction.  The extra division by ten above reflects a unit adjustment from 100µV to 1mV to squeeze the result into the range of a signed 16-bit integer.

The final task is to include a special 5V meter display. As before the first step is to list the new meter in the *MeterSet.h* header:

```
METER_3V3,
METER_5V,
METER_ADC,
```

A matching definition must then be added to *Meter_Descriptor* array in *MeterSet.c*. These descriptor lines define various meter attributes such as its name, value range, and exponent along as well as optional information not applicable in this particular case.

```
{ O(VUSB) "USB (V)", T_BLANK, 0, INT_MAX, 0, INT_MAX, -3, T_REF(BLANK), T_REF(BLANK), NO_GROUP,      VM_DEV, L(0,0,0,0) },
{ O(5V)   "5V (V)",  T_BLANK, 0, INT_MAX, 0, INT_MAX, -3, T_REF(BLANK), T_REF(BLANK), NO_GROUP,      VM_DEV, L(0,0,0,0) },
{ O(ADC)  "ADC (V)", T_BLANK, 0, INT_MAX, 0, INT_MAX, -4, T_REF(BLANK), T_REF(BLANK), G(ADC_Result), VM_DEV, L(0,0,0,0) },
```

Lastly the value in the *Meter_Values* must be updated on request from the service application. The *Monitor_Update* callback in *Monitor.c* lazily recalculates its set of ADC sensor readings within a sub-section:

```
M(3V3) = Analog_Get3V3();
M(5V) = Analog_Get5V();
```

Here *M(5V)* is a short-hand for *Meter_Values[METER_5V]* as provided by a utiltity macro. A similar P(n) construct is used to save on typing for the frequently used parameter values.

## 3.2. PWM regulation

The second example task is to implement a basic controller for the PWM signals driving the main buck-boost DC-DC regulator on the *Genius* board between DClink and the battery.

It is assumed that the pattern in the previous example has been followed for the voltage measurements *ADC_DCLINK_U* and *ADC_330V_U*, exposing the real values through *Analog_GetDCLinkVoltage()* and *Analog_GetBatteryVoltage()* functions as 32-bit integers in millivolts.

In addition the high/low-side switching current should be measured and processed, as filtered currents in milliamps as *Analog_GetLowSideCurrent()* and *Analog_GetHighSideCurrent()*.

*Note:* At this stage the current monitors have yet to be tested and it is unclear what utilizing them for this application would entail. This may involve calibration of the zero-point, a change of analog reference or synchronous conversion in phase with the switching.

The first task is to take control over the high-frequency PWM outputs. The buck-conversion requires independent control over the low and high-side duty cycles. To achieve this a configuration variable must be updated in *Config.h*:

```
#define CFG_PWM_INDEPENDENT_MODE 1
```

*Note:* Applying this options disables automatic PWM stops on debugger breakpoints. Worse, the system offers virtually no redundant protection mechanisms in hardware. Please take care to install external fuses during testing and expect frequent failures.

The PWM module controlls two half-bridge pairs for the battery and solar DC-DC converters. Let us assign them symbolic names in *PWM.h* for future reference:

```
#define PWM_BATTERY            0
#define PWM_SOLAR              1
#define NUMBER_OF_PWM_CHANNELS 2
```

In this mode the PWM module is programmed by applying a low-side, high-side and dead-time per channel along with a period for both in units of 128 MHz cycles.

For instance the example below attempts to configure the battery half-bridge for a 50%/50% duty cycle at 100 kHz minus a 16% dead-time in between. Note that these settings are buffered to avoid accidental overlap during transitions and instead the new parameters take effect simultaneously at the next *PWM_Run* update.

```
PWM_SetDeadtime(PWM_BATTERY, 100);
PWM_SetLowDuty(PWM_BATTERY, 640);
PWM_SetHighDuty(PWM_BATTERY, 640);
PWM_SetPeriod(PWM_BATTERY, 1280);
```

However before proceeding further with the regulator let us first include a few safety checks on the voltage/current ranges.

The system include an error handling module through which underlying fault conditions are reported. Declare the following new conditions in *ErrorSet.h*:

```
/******** Errors ********/
E_BUCKBOOST_CURRENT,
E_HIGH_DCLINK_VOLTAGE,
E_HIGH_BATTERY_VOLTAGE,
E_3V3_SUPPLY,
```

Next, describe include error definitions in the corresponding *Error_Descriptor* array entries in *ErrorSet.c.* The hysteresis values are timer periods, or indirect references to timing parameters, in milliseconds defining for how long a condition must be asserted for an error to be raised or cleared. The *Mode* is a user-defined bitfield defining the action to be taken while it is in effect, in this case forcing the application state-machine to switch to error mode:

```
{
        /* Order            */ ERROR_ORDER(E_BUCKBOOST_CURRENT)
        /* Name             */ "Buck-boost current",
        /* Mode             */ EM_STOP_OPERATION,
        /* RaiseHysteresis  */ 0,
        /* ClearHysteresis  */ TIMER_PARAM(ERROR_RECOVERY)
},
{
        /* Order            */ ERROR_ORDER(E_HIGH_DCLINK_VOLTAGE)
        /* Name             */ "High DC-link voltage",
        /* Mode             */ EM_STOP_OPERATION,
        /* RaiseHysteresis  */ 0,
        /* ClearHysteresis  */ TIMER_PARAM(ERROR_RECOVERY)
},
{
        /* Order            */ ERROR_ORDER(E_HIGH_BATTERY_VOLTAGE)
        /* Name             */ "High battery voltage",
        /* Mode             */ EM_STOP_OPERATION,
        /* RaiseHysteresis  */ 0,
        /* ClearHysteresis  */ TIMER_PARAM(ERROR_RECOVERY)
},
{
        /* Order            */ ERROR_ORDER(E_3V3_SUPPLY)
        /* Name             */ "3.3V supply",
        /* Mode             */ EM_STOP_OPERATION,
        /* RaiseHysteresis  */ TIMER_PARAM(3V3_VOLTAGE_HYST),
        /* ClearHysteresis  */ TIMER_PARAM(ERROR_RECOVERY)
},
```

The errors have now been declared however they still need to be tested for. The usual principle is to provide a simple extreme value test against fixed limits in the general analog processing section as a fail-safe, then add more complex tests in the higher-order logic.

Add the following error checks to *Analog_Run* in *Analog.c:*

```
Error_SetTo(E_HIGH_DCLINK_VOLTAGE, Analog_GetDCLinkVoltage() > 600000);
Error_SetTo(E_HIGH_BATTERY_VOLTAGE, Analog_GetBatteryVoltage() > 350000);
if(labs(Analog_GetLowSideCurrent()) > 10000 || labs(Analog_GetHighSideCurrent()) > 10000)
        Error_Raise(E_BUCKBOOST_CURRENT);
else
        Error_Clear(E_BUCKBOOST_CURRENT);
```

In practice such limits are rarely hard-coded and will likely be exposed as tweakable parameters through *ParameterSet.h* or as compile-time constants in *Config.h*.

The next step is to create our place-holder buck-boost controller module. Create a new *BuckBoost.h* header file and include it in the project:

```
/*****************************************************************************
 * Control the buck-boost DC-DC regulator between the battery and DC-link rails
 *****************************************************************************/
#ifndef BUCKBOOST_H
#define BUCKBOOST_H
#include "Common.h"

// Prototypes
void BuckBoost_Init(void);
void BuckBoost_Run(void);
void BuckBoost_SetStop();
void BuckBoost_SetBuck(int32_t batteryVoltage, int32_t batteryCurrent);
void BuckBoost_SetBoost(int32_t dclinkVoltage);

#endif
```

This interface allows the caller to request either buck or boost conversion mode, for constant current/voltage charging or for generating a fixed DClink rail voltage. Obviously such a simplistic scheme is unsuitable for the final application but this example is intended to demonstrate the basic principles of using the system and not to define the regulation algorithm.

Before implementing the controller let us integrate the module into the rest of the system. First, make certain that it gets initialized and updated once per cycle from the main loop. Add the following lines to *Main.c*:

```
#include "Application.h"
#include "BuckBoost.h"
#include "Common.h"

Shutdown_Init();
PWM_Init();
BuckBoost_Init();

// Update the interval timers
Timer_Run();
Trap();
// Update the buck-boost controller
BuckBoost_Run();
```

Furthermore the old test code presently attempts to program user-specified test values into the PWM channels. In the future a test override parameter may be used to select their use but for now we shall simply remove them:

```
                // Refresh the PWM outputs
#               if NUMBER_OF_PWM_CHANNELS != 2
#                       error
#               endif
                PWM_SetPeriod(P(PWM_PERIOD));
#               if CFG_PWM_INDEPENDENT_MODE
                PWM_SetDeadtime(P(PWM_DEADTIME));
                PWM_SetLowDuty(0, P(PWM_DUTY_L1));
                PWM_SetHighDuty(0, P(PWM_DUTY_H1));
                PWM_SetLowDuty(1, P(PWM_DUTY_L2));
```

```
                PWM_SetHighDuty(1, P(PWM_DUTY_H2));
#               else
                PWM_SetLowsideDeadtime(P(PWM_LOWSIDE_DEADTIME));
                PWM_SetHighsideDeadtime(P(PWM_HIGHSIDE_DEADTIME));
                PWM_SetEnable(0, P(PWM_ENABLE1));
                PWM_SetDuty(0, P(PWM_DUTY1));
                PWM_SetEnable(1, P(PWM_ENABLE2));
                PWM_SetDuty(1, P(PWM_DUTY2));
#               endif
                PWM_Run();
                Trap();
```

Next, activate the boost converter to generate 500V on DC-link from within the operating state of the main application state-machine. Upon leaving the *APP_OPERATING* state, such as during an error, the PWM enable signal will automatically be switched off.

```
#include "Application.h"
#include "BuckBoost.h"
#include "Ctrl.h"

// Main operating mode
case APP_OPERATING:
        App_Relay.PWM = true;
        BuckBoost_SetBoost(500000);
        break;

App_RelayControl(PWM, PWM, CTRL_PWM_ENABLE);
if(!App_Relay.PWM)
        BuckBoost_SetStop();
```

Finally we may implement our place-holder regulator in the new *BuckBoost.c* file:

```
#include "Analog.h"
#include "Common.h"
#include "BuckBoost.h"
#include "Parameter.h"
#include "PWM.h"
#include "Timer.h"

static enum { BUCKBOOST_STOP, BUCKBOOST_BUCK, BUCKBOOST_BOOST } BuckBoost_Mode, BuckBoost_LastMode;
static int32_t BuckBoost_TargetVoltage, BuckBoost_TargetCurrent;
static TimerMS_t BuckBoost_Timer;

/******************************************************************************
 * Reset the module
 ******************************************************************************/
void BuckBoost_Init(void)
{
    BuckBoost_SetStop();
}

/******************************************************************************
```

```
 * Perform the most recently set regulation function
 ****************************************************************************/
void BuckBoost_Run(void)
{
    uint16_t low = 0;
    uint16_t high = 0;

    // Reset the duty-cycles on a mode change
    if(BuckBoost_LastMode == BuckBoost_Mode)
    {
        // Perform the desired regulation function
        switch(BuckBoost_Mode)
        {
            // Implement me!
            /*
        case BUCKBOOST_BUCK:
             */
        case BUCKBOOST_BOOST:
            // Modulate the low-side for boost conversion, increasing or
            // decreasing it at a fixed rate until the target voltage has been
            // reached
            low = PWM_GetLowDuty(PWM_BUCKBOOST);
            if(Timer_PeriodicMS(&BuckBoost_Timer, 100))
            {
                if(Analog_GetDCLinkVoltage() < BuckBoost_TargetVoltage)
                    ++low;
                else if(low)
                    --low;
            }
            break;
        }
    }
    BuckBoost_LastMode = BuckBoost_Mode;

    // Apply the new signal, with the user-specified frequency and dead-time
    PWM_SetLowDuty(PWM_BUCKBOOST, low);
    PWM_SetHighDuty(PWM_BUCKBOOST, high);
    PWM_SetPeriod(P(PWM_PERIOD));
    PWM_SetDeadtime(P(PWM_DEADTIME));
}

/****************************************************************************
 * Latch in a desired operating mode an target voltage/current levels
 ****************************************************************************/
void BuckBoost_SetBuck(int32_t batteryVoltage, int32_t batteryCurrent)
{
    BuckBoost_Mode = BUCKBOOST_BUCK;
    BuckBoost_TargetVoltage = batteryVoltage;
```

```
    BuckBoost_TargetCurrent = batteryCurrent;
}
void BuckBoost_SetBoost(int32_t dclinkVoltage)
{
    BuckBoost_Mode = BUCKBOOST_BOOST;
    BuckBoost_TargetVoltage = dclinkVoltage;
}
void BuckBoost_SetStop(void)
{
    BuckBoost_Mode = BUCKBOOST_STOP;
}
```